

# DungeonKUrowler

## Design D1 Updated Sections

|               |                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------|
| <b>Source</b> | Design D1.pdf pages updated from the current codebase                                           |
| <b>Scope</b>  | Communication diagrams, UML class diagram, design alternatives, ranged combat, armor, rendering |
| <b>Output</b> | Word/PDF-ready D1 update document                                                               |

This document updates the previous D1 pages for the current codebase. It replaces the old controller/UI names in the original PDF with the classes that are now implemented, and adds the ranged-combat and armor behavior that was added later.

## Communication Diagrams

### 1. Start Game

Current flow: the main menu no longer starts a dungeon directly in every case. **START GAME** enters the tower flow. If no save exists, a new tower run is created; if saves exist, the player chooses a save first.

mermaid block

```
sequenceDiagram
    actor Player
    participant MainMenuWindow
    participant SaveGameController
    participant TowerProgressController
    participant LoadGameDialog
    participant TowerSessionController

    Player->>MainMenuWindow: click START GAME
    MainMenuWindow->>SaveGameController: listSaves()
    alt no saves
        MainMenuWindow->>TowerProgressController: startNewRun()
        MainMenuWindow->>TowerSessionController: startFrom(progress)
    else saves exist
        MainMenuWindow->>LoadGameDialog: show(saves)
        LoadGameDialog-->>MainMenuWindow: selected save / delete / cancel
        MainMenuWindow->>TowerProgressController: loadFromSave(save)
        MainMenuWindow->>TowerSessionController: startFrom(progress)
    end
end
```

Key classes: **MainMenuWindow**, **SaveGameController**, **TowerProgressController**, **LoadGameDialog**, **TowerSessionController**.

### 2. Build Map

Current flow: **DesignWindow** is the build-mode UI. It delegates map mutation to **BuildModeController**, which delegates validation/placement to **StandardBuildPlacementStrategy**.

mermaid block

```
sequenceDiagram
    actor Player
    participant DesignWindow
    participant BuildModeController
    participant StandardBuildPlacementStrategy
    participant LockedChestKeyPlacer
    participant DungeonMap

    Player->>DesignWindow: select tool
```

```

DesignWindow->>BuildModeController: selectTool(tool)
Player->>DesignWindow: click / drag grid cell
DesignWindow->>BuildModeController: placeSelectedToolAt(x, y)
BuildModeController->>StandardBuildPlacementStrategy: place(map, x, y, tool)
StandardBuildPlacementStrategy->>DungeonMap: mutate GridCell
alt placed locked chest
    BuildModeController->>LockedChestKeyPlacer: assignAndPlace(map, chest)
end
BuildModeController-->>DesignWindow: placed + optional message
DesignWindow->>DesignWindow: repaint canvas / refresh selected label

```

Key classes: **DesignWindow**, **BuildModeController**, **BuildToolCatalog**, **BuildTool**, **StandardBuildPlacementStrategy**, **LockedChestKeyPlacer**, **DungeonMap**, **GridCell**.

### 3. Load Map

There are two load-map paths now: load a designed map directly from the main menu, or load/save inside build mode.

#### mermaid block

```

sequenceDiagram
    actor Player
    participant MainMenuWindow
    participant BuildMapFileDialog
    participant BuildModeController
    participant BuildMapPersistence
    participant GameEngine
    participant GameWindow

    Player->>MainMenuWindow: click LOAD MAP
    MainMenuWindow->>BuildMapFileDialog: showLoad()
    BuildMapFileDialog-->>MainMenuWindow: selected Path
    MainMenuWindow->>BuildModeController: new controller
    MainMenuWindow->>BuildModeController: loadMap(path)
    BuildModeController->>BuildMapPersistence: load(path)
    BuildMapPersistence-->>BuildModeController: DungeonMap
    MainMenuWindow->>GameEngine: new GameEngine(designMap)
    MainMenuWindow->>GameWindow: new GameWindow(engine)

```

Build-mode save/load uses the same persistence service:

#### mermaid block

```

sequenceDiagram
    actor Player
    participant DesignWindow
    participant BuildMapFileDialog
    participant BuildModeController
    participant BuildMapPersistence

    Player->>DesignWindow: click SAVE or LOAD
    DesignWindow->>BuildMapFileDialog: showSave() / showLoad()
    alt save
        DesignWindow->>BuildModeController: saveMap(path)
        BuildModeController->>BuildMapPersistence: save(designMap, path)
    else load
        DesignWindow->>BuildModeController: loadMap(path)
        BuildModeController->>BuildMapPersistence: load(path)
        DesignWindow->>DesignWindow: repaint canvas
    end
end

```

Key classes: **BuildMapFileDialog**, **BuildModeController**, **BuildMapPersistence**, **MainMenuWindow**, **DesignWindow**.

### 4. Add Random Items

Current flow: the old **RandomItemGenerator** is now **BuildRandomItemPlacer**. The controller limits this action to three uses per map with **MAX\_RANDOM\_ITEM\_ADDS = 3**.

#### mermaid block

```

sequenceDiagram

```

```

actor Player
participant DesignWindow
participant BuildModeController
participant BuildRandomItemPlacer
participant BuildToolCatalog
participant BuildPlacementStrategy
participant DungeonMap

Player->>DesignWindow: click ADD 5 RANDOM ITEMS
DesignWindow->>BuildModeController: addFiveRandomItems()
BuildModeController->>BuildModeController: canAddFiveRandomItems()
alt remaining uses available
  BuildModeController->>BuildRandomItemPlacer: addFiveRandomItemsAndHiddenSearchable(map)
  BuildRandomItemPlacer->>BuildToolCatalog: choose item/searchable tools
  loop each generated placement
    BuildRandomItemPlacer->>BuildPlacementStrategy: place(map, x, y, tool)
    BuildPlacementStrategy->>DungeonMap: mutate GridCell
  end
  BuildModeController-->>DesignWindow: Result(placed, true)
else limit reached
  BuildModeController-->>DesignWindow: Result(0, false)
end
DesignWindow->>DesignWindow: refreshRandomItemsButton() + repaint()

```

Key classes: **BuildModeController**, **BuildRandomItemPlacer**, **BuildToolCatalog**, **BuildPlacementStrategy**, **DungeonMap**.

## 5. Run Map in Play Mode

Current flow: **DesignWindow** validates/uses its design map by constructing a **GameEngine**, then opens **GameWindow**. **GameWindow** creates the player-facing controllers and **GamePanel**.

### mermaid block

```

sequenceDiagram
    actor Player
    participant DesignWindow
    participant TeamMatchController
    participant GameEngine
    participant TargetItemMission
    participant FogOfWarEngine
    participant GameWindow
    participant GamePanel
    participant PlayerModeController
    participant InteractionController

    Player->>DesignWindow: click RUN IN PLAY MODE
    DesignWindow->>GameEngine: new GameEngine(designMap)
    GameEngine->>GameEngine: findHeroStart(map)
    GameEngine->>GameEngine: placeHeroOnMap()
    GameEngine->>FogOfWarEngine: revealAround(map, hero)
    GameEngine->>TargetItemMission: start target mission
    GameEngine->>GameEngine: startGameTimers()
    DesignWindow->>GameWindow: new GameWindow(engine)
    GameWindow->>PlayerModeController: new PlayerModeController(engine)
    GameWindow->>InteractionController: new InteractionController(engine)
    GameWindow->>GamePanel: new GamePanel(engine, controllers)

```

Team Match is a separate run option:

### mermaid block

```

sequenceDiagram
    actor Player
    participant DesignWindow
    participant TeamMatchController
    participant TeamMatchGameFactory
    participant TeamMatchTeamPlacer
    participant GameEngine

    Player->>DesignWindow: click RUN TEAM MATCH
    DesignWindow->>TeamMatchController: startFromDesignMap(map)
    TeamMatchController->>TeamMatchTeamPlacer: place teams
    TeamMatchController->>TeamMatchGameFactory: create engine
    TeamMatchGameFactory->>GameEngine: createTeamMatch(map, hero)

```

Key classes: **DesignWindow**, **GameEngine**, **GameWindow**, **GamePanel**, **PlayerModeController**, **InteractionController**, **TeamMatchController**.

## 6. Move Hero

Current flow: movement is keyboard-driven from **GamePanel**, but movement rules live in **PlayerModeController** and **GameEngine**.

### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant PlayerModeController
    participant GameEngine
    participant Hero
    participant DungeonMap
    participant FogOfWarEngine

    Player->>GamePanel: press WASD / arrow key
    GamePanel->>PlayerModeController: moveHero(direction)
    PlayerModeController->>GameEngine: canHeroAct()
    PlayerModeController->>Hero: getEnergy()
    PlayerModeController->>DungeonMap: isCellPassable(nx, ny)
    alt valid move and energy >= 5
        PlayerModeController->>Hero: consumeEnergy(5)
        PlayerModeController->>GameEngine: updateHeroPosition(nx, ny)
        GameEngine->>DungeonMap: remove hero from old cell / add to new cell
        GameEngine->>Hero: updatePosition(nx, ny)
        GameEngine->>FogOfWarEngine: revealAround(map, hero)
        GameEngine->>GameEngine: notifyListeners()
    else invalid move
        PlayerModeController-->>GamePanel: no state change
    end
end
```

Key classes: **GamePanel**, **PlayerModeController**, **GameEngine**, **Hero**, **DungeonMap**, **FogOfWarEngine**.

## 7. Interact with Object

Current flow: mouse click selects a visible grid cell. **GamePanel** first attempts combat on the clicked cell, then asks **InteractionController** for item interactions.

### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant CombatController
    participant InteractionController
    participant DungeonMap
    participant GridCell
    participant ItemActionMenuDialog
    participant GameEngine

    Player->>GamePanel: click visible grid cell
    GamePanel->>CombatController: attackAt(gridX, gridY)
    alt attack succeeds
        CombatController->>GameEngine: apply melee/ranged combat
        GamePanel->>GamePanel: leaveDefeatMarker if defeated
    else no attack target
        GamePanel->>InteractionController: getItemInteractions(gridX, gridY)
        InteractionController->>DungeonMap: isHeroAdjacent(hero, x, y)
        InteractionController->>GridCell: getItemView()
        InteractionController->>GamePanel: ItemInteraction list
        GamePanel->>ItemActionMenuDialog: show action menu
        ItemActionMenuDialog->>GamePanel: selected action
        GamePanel->>InteractionController: applyGroundAction(item, x, y, action)
        InteractionController->>GameEngine: search / break / take item / performInventoryAction
    end
end
```

Key classes: `GamePanel`, `CombatController`, `InteractionController`, `ItemActionMenuDialog`, `GameEngine`, `DungeonMap`.

## 8. Collect Item

Current flow: item pickup is shared by keyboard (T), click interaction, container loot, and search results. Coins and valuables bypass the 8-slot per-level bag.

### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant InteractionController
    participant InventoryController
    participant GameEngine
    participant DungeonMap
    participant Hero
    participant Inventory
    participant FullGameInventory

    Player->>GamePanel: press T or choose Take/Collect
    alt keyboard pickup
        GamePanel->>GameEngine: takeItemOnGround()
    else menu pickup
        GamePanel->>InteractionController: takeItemAt(x, y)
        InteractionController->>InventoryController: takeFirstItemFromCell(x, y)
    end
    InventoryController->>DungeonMap: isHeroAdjacent(hero, x, y)
    InventoryController->>DungeonMap: read first takable item
    alt Coin
        InventoryController->>GameEngine: takeItem(coin, x, y)
        GameEngine->>Hero: earnCoins(value)
    else ValuableItem
        InventoryController->>GameEngine: takeItem(valuable, x, y)
        GameEngine->>FullGameInventory: add(valuable)
    else normal Item
        InventoryController->>Inventory: hasFreeSlot()
        InventoryController->>GameEngine: takeItem(item, x, y)
        GameEngine->>Inventory: tryAdd(item)
    end
    GameEngine->>DungeonMap: removeItemFromCell(item, x, y)
    GameEngine->>GameEngine: checkTargetMissionPickup() + fireItemPickedUp() + notifyListeners()
```

Key classes: `GamePanel`, `InteractionController`, `InventoryController`, `GameEngine`, `Hero`, `Inventory`, `FullGameInventory`.

## 9. Manage Inventory

Current flow: inventory UI lists carried items and their actions. The effect of each action is delegated through `ItemActionEffects`, which is a command/strategy registry.

### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant InventoryDialog
    participant Hero
    participant Inventory
    participant GameEngine
    participant ItemActionEffects
    participant Item

    Player->>GamePanel: press I / click inventory button
    GamePanel->>InventoryDialog: new InventoryDialog(owner, engine)
    InventoryDialog->>Hero: getInventory()
    InventoryDialog->>Inventory: getItems()
    InventoryDialog->>Item: getInventoryActions()
    InventoryDialog->>Player: show DRINK / WEAR / EQUIP / READ / DISCARD / REMOVE
    Player->>InventoryDialog: choose action
    InventoryDialog->>GameEngine: performInventoryAction(item, action)
    GameEngine->>ItemActionEffects: forAction(action)
    ItemActionEffects->>Hero: drink / wearArmor / wearRing / equipWeapon / removeEquipment / discard
```

GameEngine->>GameEngine: reveal fog + notifyListeners if action changed state

### Armor-specific path:

#### mermaid block

```
sequenceDiagram
    actor Player
    participant InventoryDialog
    participant GameEngine
    participant ItemActionEffects
    participant Hero
    participant Armor
    participant GamePanel

    Player->>InventoryDialog: choose WEAR or EQUIP on Armor
    InventoryDialog->>GameEngine: performInventoryAction(armor, WEAR/EQUIP)
    GameEngine->>ItemActionEffects: WearEffect / EquipEffect
    ItemActionEffects->>Hero: wearArmor(armor)
    Hero->>Armor: getDefModifier()
    GamePanel->>Hero: getEquippedArmor()
    GamePanel->>HeroArmorPixelArt: paintEquipped(...)
```

Key classes: **InventoryDialog**, **GameEngine**, **ItemActionEffects**, **Hero**, **Armor**, **Weapon**, **Ring**, **Inventory**.

## 10. Fight Enemy

Current flow: combat supports melee, enemy projectiles, and hero-owned ranged projectiles.

### 10A. Melee Attack

#### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant CombatController
    participant GameEngine
    participant CombatManager
    participant Hero
    participant Enemy
    participant GridCell

    Player->>GamePanel: click enemy cell or press P
    GamePanel->>CombatController: attackAt(x, y) / attackNearestEnemy()
    CombatController->>GameEngine: canHeroAct()
    CombatController->>GameEngine: isHeroAttackOnCooldown()
    CombatController->>Hero: getEquippedWeapon()
    alt melee weapon or unarmed
        CombatController->>DungeonMap: isHeroAdjacent(hero, x, y)
        CombatController->>GameEngine: firstHostileInCell(cell)
        CombatController->>CombatManager: heroAttacksKnight/Sorcerer/Boss(hero, target)
        CombatManager->>Hero: consumeEnergy(2)
        CombatManager->>CombatManager: generateDamage() + receiveDamage()
        CombatManager->>Enemy: setHp(hpAfter)
        CombatController->>GameEngine: recordHeroAttackPacing()
        CombatController->>GameEngine: fireHeroAttack(result)
        alt defeated
            CombatController->>GridCell: remove(target)
            CombatController->>GameEngine: fireEnemyDefeated(target)
        end
    end
    CombatController->>GameEngine: notifyGameStateChanged()
end
```

### 10B. Ranged Attack

Current ranged weapons:

| Weapon | Category | ATK | Range setting | Cost | Projectile |
|--------|----------|-----|---------------|------|------------|
|--------|----------|-----|---------------|------|------------|

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| --- | --- | --- | --- | --- | --- |
|-----|-----|-----|-----|-----|-----|

| Wooden Bow | bows | 6 | **WeaponType.maxRange = 4**; current test build ignores range in **canHeroRangedTarget** and keeps line-of-sight only | 3 Energy | Arrow |

| Magic Wand | staves | 8 | **WeaponType.maxRange = 4**; current test build ignores range in **canHeroRangedTarget** and keeps line-of-sight only | 5 Mana | Ice bolt |

#### mermaid block

```
sequenceDiagram
    actor Player
    participant GamePanel
    participant CombatController
    participant GameEngine
    participant CombatManager
    participant Hero
    participant Weapon
    participant Projectile
    participant Enemy

    Player->>GamePanel: press P with ranged weapon
    GamePanel->>CombatController: autoAimRangedAttack()
    CombatController->>Hero: getEquippedWeapon()
    CombatController->>GameEngine: canHeroShootAt(x, y)
    GameEngine->>GameEngine: canHeroRangedTarget(x, y)
    GameEngine->>GameEngine: straight ray + clear projectile path
    CombatController->>GameEngine: launchHeroRangedAttackAt(bestX, bestY)
    GameEngine->>CombatManager: prepareHeroRangedProjectile(hero, target)
    CombatManager->>Weapon: getAtkValue(), getRangedCostType(), getRangedCostAmount()
    alt enough Energy/Mana
        CombatManager->>Hero: spendEnergy(3) or spendMana(5)
        CombatManager-->>GameEngine: HeroProjectilePrep(damage, style)
        GameEngine->>Projectile: new Projectile(heroOwned=true, style)
        GameEngine->>GameEngine: recordHeroAttackPacing()
    else insufficient resource
        CombatManager-->>GameEngine: null
    end
    end
    loop projectile timer
        GameEngine->>GameEngine: advanceHeroProjectile(projectile)
        alt hostile in current or next cell
            GameEngine->>CombatManager: applyHeroProjectileHit(target, prep)
            CombatManager->>Enemy: setHp(hpAfter)
            GameEngine->>GameEngine: fireHeroAttack / fireEnemyDefeated / notifyListeners
        end
    end
    end
```

#### Enemy projectile path:

#### mermaid block

```
sequenceDiagram
    participant GameEngine
    participant SorcererOrBoss
    participant CombatManager
    participant Projectile
    participant Hero

    GameEngine->>GameEngine: updateSorcererAttacks() / updateBossAttacks()
    GameEngine->>GameEngine: hasClearProjectilePath(enemy, hero, range)
    GameEngine->>CombatManager: prepareSorcererProjectile() / prepareBossProjectile()
    CombatManager->>SorcererOrBoss: spend mana
    GameEngine->>Projectile: spawnProjectile(heroOwned=false)
    loop projectile timer
        GameEngine->>GameEngine: advanceEnemyProjectile(projectile)
        alt hits hero
            GameEngine->>CombatManager: applyProjectileImpact(hero, damage)
            CombatManager->>Hero: setHp(hpAfter)
            GameEngine->>GameEngine: fireHeroTookDamage / finishGameIfNeeded
        end
    end
    end
```

Key classes: **CombatController**, **CombatManager**, **GameEngine**, **Projectile**, **Weapon**, **WeaponType**, **HeroProjectileStyle**, **RangedCostType**, **Hero**, **Knight**, **Sorcerer**, **BossEnemy**.

## UML Class Diagram - Updated Class List and Relationships

The original PDF class diagram used older placeholders such as **GameController**, **MapEditorController**, **BuildMapUI**, **PlayModeUI**, **Map**, and generic **Enemy**. The current implementation has more specific classes and separates UI, controllers, services, model objects, rendering, and persistence.

#### mermaid block

```
classDiagram
    class MainMenuWindow {
        +handleStartGame()
        +handleLoadGame()
        +loadSavedMap()
        +openTowerMap(progress)
    }

    class DesignWindow {
        -BuildModeController controller
        -TeamMatchController teamMatchController
        +saveMap()
        +loadMap()
        +runInPlayMode()
        +runTeamMatch()
    }

    class GameWindow {
        -GameEngine engine
        +onGameStateChanged()
    }

    class GamePanel {
        -PlayerModeController playerModeController
        -InteractionController interactionController
        -CombatController combatController
        +onGameStateChanged()
        +paintComponent(g)
    }

    class BuildModeController {
        -DungeonMap designMap
        -BuildTool selectedTool
        +selectTool(tool)
        +placeSelectedToolAt(x, y)
        +eraseAt(x, y)
        +saveMap(path)
        +loadMap(path)
        +addFiveRandomItems()
    }

    class BuildMapPersistence {
        +save(map, path)
        +load(path) DungeonMap
    }

    class BuildRandomItemPlacer {
        +addFiveRandomItemsAndHiddenSearchable(map)
    }

    class BuildPlacementStrategy {
        <<interface>>
        +place(map, x, y, tool) boolean
        +erase(map, x, y) boolean
    }

    class StandardBuildPlacementStrategy

    class GameEngine {
        -DungeonMap dungeonMap
        -Hero hero
        -CombatManager combatManager
        -FogOfWarEngine fogEngine
        -TargetItemMission targetMission
        -List~Projectile~ activeProjectiles
        +updateHeroPosition(nx, ny)
        +takeItem(item, x, y)
        +performInventoryAction(item, action)
        +launchHeroRangedAttackAt(x, y)
        +canHeroShootAt(x, y)
        +tryUnlock(target)
        +search(object)
    }
```



```

    +notifyListeners()
}

class PlayerModeController {
    -GameEngine engine
    +moveHero(direction)
    +consumePotion()
    +consumePotionOnGround()
}

class InteractionController {
    -GameEngine engine
    -InventoryController inventoryController
    -BreakController breakController
    +getItemInteractions(x, y)
    +takeItemAt(x, y)
    +applyGroundAction(item, x, y, action)
    +breakNearestObject()
    +search(object)
}

class InventoryController {
    -GameEngine engine
    +takeFirstItemFromCell(x, y) PickupResult
}

class CombatController {
    -GameEngine engine
    -CombatManager combatManager
    +attackAt(x, y)
    +attackNearestEnemy()
    +autoAimRangedAttack()
}

class CombatManager {
    +heroAttacksKnight(hero, knight)
    +heroAttacksSorcerer(hero, sorcerer)
    +heroAttacksBoss(hero, boss)
    +prepareHeroRangedProjectile(hero, target)
    +applyHeroProjectileHit(target, prep)
    +prepareSorcererProjectile(attacker, hero)
    +prepareBossProjectile(attacker, hero)
}

class ItemActionEffects {
    +forAction(action) Effect
}

class DungeonMap {
    -GridCell[][] cells
    +getCell(x, y)
    +isCellPassable(x, y)
    +isHeroAdjacent(hero, x, y)
    +removeItemFromCell(item, x, y)
}

class GridCell {
    -List~Item~ items
    -List~Entity~ entities
    +isPassable()
    +isWalkable()
    +getItemView()
    +getEntitiesView()
}

class Entity {
    #int x
    #int y
    #String name
    +spriteResource()
}

class Hero {
    -Inventory inventory
    -FullGameInventory fullInventory
    -Armor equippedArmor
    -Weapon equippedWeapon
    -Ring equippedRing
    -long lastAttackTimeMs
    +getDef()
}

```

```

        +wearArmor(armor)
        +equipWeapon(weapon)
        +wearRing(ring)
        +spendEnergy(amount)
        +spendMana(amount)
    }

    class Knight
    class Sorcerer
    class BossEnemy

    class Item {
        +getInventoryActions()
        +isTakable()
        +spriteResource()
    }

    class Weapon {
        -WeaponType type
        +getAtkValue()
        +isRanged()
        +getMaxRange()
        +getRangedCostType()
        +getProjectileStyle()
    }

    class WeaponType {
        <<record>>
        +id
        +baseAttack
        +ranged
        +maxRange
        +rangedCostType
        +rangedCostAmount
        +projectileStyle
    }

    class WeaponCatalog {
        +byId(id)
        +randomIn(category, rng)
    }

    class Armor {
        -int defModifier
        +getDefModifier()
        +getInventoryActions()
        +spriteResource()
    }

    class Ring
    class Projectile
    class Inventory
    class FullGameInventory
    class FogOfWarEngine
    class SpriteRegistry
    class AssetManager
    class HeroArmorPixelArt

    MainMenuWindow --> SaveGameController
    MainMenuWindow --> TowerProgressController
    MainMenuWindow --> TowerSessionController
    DesignWindow --> BuildModeController
    DesignWindow --> GameEngine
    DesignWindow --> TeamMatchController
    GameWindow --> GamePanel
    GamePanel --> PlayerModeController
    GamePanel --> InteractionController
    GamePanel --> CombatController
    BuildModeController --> BuildMapPersistence
    BuildModeController --> BuildRandomItemPlacer
    BuildModeController --> BuildPlacementStrategy
    BuildPlacementStrategy <|.. StandardBuildPlacementStrategy
    GameEngine --> DungeonMap
    GameEngine --> Hero
    GameEngine --> CombatManager
    GameEngine --> FogOfWarEngine
    GameEngine --> Projectile
    PlayerModeController --> GameEngine
    InteractionController --> InventoryController
    InteractionController --> BreakController

```

```
InventoryController --> GameEngine
CombatController --> GameEngine
CombatController --> CombatManager
DungeonMap --> GridCell
GridCell --> Item
GridCell --> Entity
Entity <|-- Hero
Entity <|-- Knight
Entity <|-- Sorcerer
Entity <|-- BossEnemy
Item <|-- Weapon
Item <|-- Armor
Item <|-- Ring
Weapon --> WeaponType
WeaponCatalog --> WeaponType
Hero --> Inventory
Hero --> FullGameInventory
Hero --> Armor
Hero --> Weapon
Hero --> Ring
SpriteRegistry --> AssetManager
GamePanel --> SpriteRegistry
GamePanel --> HeroArmorPixelArt
```

# Design Alternatives Discussion

## Current Design

The current design is controller/service oriented and closer to GRASP than the earlier D1 draft. UI classes forward user intent; controllers coordinate use cases; model classes own state; service/factory/catalog classes isolate creation, persistence, combat math, rendering assets, and rules.

Current responsibilities:

- **MainMenuWindow**: top-level navigation; starts tower flow, load-game flow, load-map flow, or build mode.
- **DesignWindow**: build-mode view; delegates all map edits to **BuildModeController**.
- **BuildModeController**: build-mode GRASP Controller; owns selected tool, design map, save/load, random item count, and placement requests.
- **StandardBuildPlacementStrategy**: placement rules for floors, walls, tools, items, containers, weapons, armor, and searchables.
- **BuildMapPersistence**: serializes/deserializes designed maps.
- **GameWindow** and **GamePanel**: gameplay shell and observer/rendering surface. They forward key/mouse input to controllers and repaint after **GameEngine** notifications.
- **GameEngine**: central game-state owner and observer subject. It owns map, hero, timers, mission state, fog, projectiles, enemy spawning/AI ticks, pickup/search/open logic, and listener notifications.
- **PlayerModeController**: movement use case; checks energy and passability before asking **GameEngine** to move the hero.
- **InteractionController**: ground item interaction, search, break, and take/use-on-ground flow.
- **InventoryController**: pickup validation and item transfer from map to inventory/full inventory.
- **CombatController**: player-initiated melee/ranged combat and auto-aim.
- **CombatManager**: stateless combat formulas, resource spending for ranged attacks, projectile prep, and damage application.
- **ItemActionEffects**: command/strategy registry for inventory actions (**DRINK**, **WEAR**, **EQUIP**, **READ**, **REMOVE**, **DISCARD**, **SEARCH**, **BREAK**, **OPEN**, etc.).
- **WeaponCatalog** / **WeaponType**: Flyweight catalog for weapon stats and sprites. B23 ranged weapons are registered here.
- **Armor**: equipment item that contributes to **Hero.getDef()** and supplies **/weapons/armor.png** as its item sprite.
- **HeroArmorPixelArt**: fitted equipped-armor overlay for the hero sprite; item icon and equipped overlay are intentionally separate.

## Pros

- Clearer separation between UI and rules than the old D1 diagram.
- Build mode uses a Strategy (**BuildPlacementStrategy**) instead of placing everything directly in the UI.
- Inventory action behavior is extensible through **ItemActionEffects** instead of a large UI switch.
- Combat math is centralized in **CombatManager**.

- Ranged weapons are data-driven through `WeaponType` and `WeaponCatalog`.
- Rendering assets are separated through `SpriteRegistry`, `AssetManager`, and sprite-resource overrides.
- Armor is integrated as normal equipment: inventory action, DEF calculation, item sprite, and equipped visual.
- `GameEngine` implements observer-style listener notification so views repaint after state changes.

## Cons / Remaining Risks

- `GameEngine` is still large. It owns core state, timers, projectiles, enemy AI updates, mission state, pickups, search, locks, fog, and notifications.
- `GamePanel` still has many rendering and input responsibilities in one class.
- Hero ranged range is currently in test mode: `WeaponType.maxRange` is 4, but `GameEngine.canHeroRangedTarget` and `CombatController.autoAimRangedAttack` currently use `Integer.MAX_VALUE` while preserving straight-line and wall/projectile-block checks.
- Some class names changed significantly from the earlier D1 diagrams, so old documentation using `GameController`, `MapEditorController`, `BuildMapUI`, `PlayModeUI`, and generic `Enemy` no longer matches the code.

## Alternative 1: Split GameEngine into Smaller Services

Move projectile ticking, enemy AI timers, mission/floor completion, and pickup/search/open logic into separate services/controllers.

Pros:

- `GameEngine` would become a thinner state facade.
- Easier unit testing for projectile movement and enemy AI.
- Lower risk when modifying one subsystem.

Cons:

- More dependency wiring.
- More classes for teammates to understand.
- Current project size may not justify full decomposition yet.

Recommended use: split next if new combat/AI/floor mechanics continue to grow.

## Alternative 2: Keep GameEngine as Facade, Extract Rendering Helpers

Keep `GameEngine` as the state owner, but move more `GamePanel` drawing into small renderers, similar to `AmbienceRenderer`.

Pros:

- Reduces `GamePanel` size.
- Makes ranged projectile, HUD, hero equipment, and item drawing easier to maintain.
- Preserves current controller/model architecture.

Cons:

- Rendering helpers need stable inputs and careful ordering.
- Does not reduce `GameEngine` complexity.

Recommended use: good near-term refactor because the current UI has grown with ranged weapons, armor, fog, pets, HUD, projectiles, and animation.

### Alternative 3: Make Ranged Weapon Rules Fully Data-Driven

Restore strict `weapon.getMaxRange()` checks and move all B23 weapon values into a data file or dedicated catalog configuration.

Pros:

- Easier balancing: bow/wand cost, damage, range, projectile style can be changed without touching combat flow.
- Documentation and code will match weapon stats exactly.
- Avoids temporary test-mode range behavior becoming accidental final behavior.

Cons:

- Needs migration or validation for old saved maps.
- More data validation required.

Recommended use: before final submission, restore dynamic max-range behavior if the requirement says weapons must be range-limited.

### Alternative 4: Treat Armor Visuals as Two Separate Assets

Current implementation uses `/weapons/armor.png` for item/build/inventory display and code-drawn fitted overlay for the equipped hero.

Pros:

- Item icon can remain high quality.
- Equipped armor does not replace or distort the hero sprite.
- Weapons still draw on top of armor because `GamePanel` draws armor first, then equipped weapon.

Cons:

- Item icon and equipped overlay are not identical.
- More manual pixel tuning may be needed if hero sprite changes.

Recommended use: keep this split. A full armor PNG is good as an item icon, but too large to draw directly on the 16x32 hero sprite.

## Pattern / GRASP Notes

- Controller: `BuildModeController`, `PlayerModeController`, `InteractionController`, `InventoryController`, `CombatController`, `TeamMatchController`.
- Information Expert: `Hero` owns equipment and derived DEF; `Inventory` owns capacity; `DungeonMap` owns cells; `GridCell` owns local items/entities.
- Low Coupling: UI delegates to controllers; action effects are isolated in `ItemActionEffects`; rendering assets are isolated from model through sprite-resource paths and registries.
- High Cohesion: `CombatManager` handles combat formulas; `BuildMapPersistence` handles map persistence; `BuildRandomItemPlacer` handles random build-mode placement.

- Strategy: `BuildPlacementStrategy`, `EnemySpawnPolicy`, `VisibilityStrategy`, `ItemActionEffects.Effect`.
- Observer: `GameEngine` notifies `GameStateListener`; audio/game events are emitted through `GameEventListener` and mission listeners.
- Flyweight: `WeaponType` stores shared weapon intrinsic state; `Weapon` instances reference it.
- Singleton/Flyweight cache: `AssetManager` caches loaded sprites.

## Implementation Facts Added Since Original D1

- Ranged weapons:
  - `Wooden Bow`: ATK 6, Energy cost 3, projectile style `ARROW`.
  - `Magic Wand`: ATK 8, Mana cost 5, projectile style `ICE_BOLT`.
  - Both are registered in `WeaponCatalog.registerB23RangedWeapons()`.
- Ranged pacing:
  - Hero attacks use `Hero.lastAttackTimeMs`.
  - `GameConstants.GLOBAL_ACTION_TICK_MS` controls the cooldown.
  - `CombatController` ignores inputs during cooldown.
- Projectiles:
  - Hero-owned projectiles and enemy-owned projectiles share `Projectile`.
  - `GameEngine.updateProjectiles()` advances them on a timer.
  - Hero projectiles check the current cell before moving, then the next cell, so moving enemies are less likely to be skipped.
- Armor:
  - `Armor.getInventoryActions()` returns `WEAR`, `EQUIP`, and `DISCARD`.
  - `ItemActionEffects.WearEffect` and `EquipEffect` both support armor.
  - `Hero.getDef()` adds base DEF + armor bonus + ring bonus.
  - `Armor.spriteResource()` returns `/weapons/armor.png`.
  - Equipped armor is drawn with `HeroArmorPixelArt.paintEquipped(...)` so it fits the hero sprite and does not cover the weapon overlay.
- Rendering:
  - No external runtime image downloads are used.
  - Existing local sprites live under `src/main/resources`.
  - Bow/wand custom pixel art is drawn in `GamePanel`, `DesignWindow`, and `InventoryDialog`.
  - Armor item icon uses the local PNG, while equipped armor uses fitted pixel overlay.